BÁO CÁO BÀI TẬP LỚN – CS112

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM

Môn: CS112 – Phân tích và thiết kế thuật toán, lớp CS112.F31

BÀI TẬP LỚN

TÔ MÀU ĐỒ THỊ (BÀI TOÁN SỞ THÚ): THIẾT KẾ, SO SÁNH NĂM CHIẾN LƯỢC GIẢI VÀ DEMO MINH HỌA TỪNG BƯỚC

Giảng viên: [SV XÁC NHẬN]

Sinh viên: [Họ tên – MSSV]

Demo trực tuyến: https://algorithm-visualizer.hieutk.dev

TP. Hồ Chí Minh, tháng 10/2026

MỤC LỤC

Chương 1. Giới thiệu bài toán.....5
1.1. Lý do chọn bài toán.....5
1.2. Nguồn gốc.....5
1.3. Ứng dụng thực tế.....6
Chương 2. Mô tả và mô hình bài toán.....7
2.1. Phát biểu theo kiểu đề thi.....7
2.2. Mô hình hóa.....7
2.3. Input.....8
2.4. Output.....8
2.5. Ràng buộc.....8
2.6. Ví dụ Input/Output.....9
Chương 3. Thiết kế cấu trúc dữ liệu và thuật toán.....11
3.1. Cấu trúc dữ liệu.....11
3.2. Phân tích đặc trưng (ý tưởng giải).....11
3.3. Năm thuật toán.....12
(1) Tham lam theo thứ tự đỉnh (Greedy).....12
(2) Welsh–Powell.....12
(3) DSATUR (Brélaz, 1979) – mở rộng ngoài sách, app ghi rõ.....13
(4) Quay lui (Levitin §12.1).....13
(5) Vét cạn.....14
Bảng tổng hợp.....15
Chế độ tìm sắc số χ(G).....15
Chương 4. Ví dụ minh họa các bước thực hiện.....17
4.1. Ví dụ chung.....17

4.2. Quay lui trên ví dụ chung (khớp đúng trace của Player).....17
4.3. Cùng ví dụ, bốn thuật toán còn lại (trace thật).....20
Chương 5. Tường thuật quá trình thiết kế và xây dựng.....24
5.1. Nói rõ ngay từ đầu.....24
5.2. Công cụ sử dụng.....24
5.3. Dòng thời gian.....25
5.4. Thiết kế chức năng và giao diện.....26
5.5. Các sự cố tiêu biểu và cách xử lý.....27
Sự cố 1 – Bài tô màu đồ thị bị review FAIL với 4 lỗi (PR #7).....27
Sự cố 2 – Nhánh cận đánh dấu sai nút tốt nhất (PR #1, bài cái túi).....28
Sự cố 3 – Lời giải tham chiếu tự tham chiếu lặp lại ở nhiều nhóm.....28
Sự cố 4 – Code viết dồn một dòng (PR #3, #4).....29
(Tuỳ chọn) Sự cố 5 – Trang So sánh cần 6 vòng review (PR #8).....29
5.6. Phần sinh viên tự kiểm chứng.....29
5.7. Đánh giá việc dùng AI.....30
Chương 6. Thiết kế thực nghiệm.....31
6.1. Cách tạo testcase.....31
6.2. Phân nhóm.....31
6.3. Đo đạc.....32
6.4. Bảo đảm độ phủ.....33
Chương 7. Đánh giá.....34
7.1. Tính đúng.....34
7.2. Kết quả so sánh.....34
7.3. Tính trực quan và khả năng minh họa.....35
7.4. Hạn chế.....35
7.5. Hướng phát triển.....36

Chương 8. Phát triển mở rộng.....37

Tài liệu tham khảo.....38

Phụ lục.....39

Chương 1. Giới thiệu bài toán

1.1. Lý do chọn bài toán

> ***[SV XÁC NHẬN]*** *Đây phải là lý do thật của bạn. Bản nháp dưới đây viết theo đúng diễn biến đã ghi lại trong lịch sử làm việc; bạn sửa lại bằng lời của mình.*

Lý do đầu tiên đến từ nhận xét của cô sau bài thi. Phần thiết kế thuật toán của nhiều bạn còn yếu, trong khi các dạng bài không mới. Cô nêu ví dụ: "Bài toán Sở thú là bài toán tô màu đồ thị". Em nhận ra mình cũng chưa trình bày tốt dạng bài này, nên muốn dùng bài tập lớn để làm lại nó cho thật kỹ.

Lý do thứ hai là gợi ý từ công cụ AI. Em cho ba mô hình AI (Claude, Codex/GPT, Grok) tranh luận về chiến lược làm bài, và cả ba cùng đề xuất chọn tô màu đồ thị làm bài trọng tâm. Lý do các mô hình đưa ra:

- bài có nhiều cách giải để so sánh: heuristic nhanh nhưng không chắc chắn, và thuật toán chính xác chắc chắn nhưng tốn thời gian lũy thừa;
- trạng thái từng bước (đỉnh đang xét, màu bị cấm, quay lui) dễ minh họa trên giao diện.

Ngoài ra, trước đó em đã xây một nền tảng demo cho nhiều thuật toán trong sách Levitin (Chương 8). Tô màu đồ thị là bài em chọn để đi sâu nhất.

1.2. Nguồn gốc

- **Bài toán bốn màu:** bài toán tô bản đồ của Guthrie (1852), được Appel và Haken chứng minh năm 1976.
- **Trong sách Levitin:**
  - §1.3 giới thiệu tô màu đồ thị như một bài toán đồ thị quan trọng;
  - §12.1 (trang 425) trình bày quay lui cho bài toán tô m màu.
- **Bài Sở thú** trong đề thi và các buổi ôn tập của môn.
- **Hai thuật toán heuristic** dùng trong bài có công bố riêng:
  - Welsh và Powell (1967);
  - DSATUR của Brélaz (1979).

1.3. Ứng dụng thực tế

- **Xếp lịch thi:** mỗi môn là một đỉnh, hai môn có chung sinh viên thì có cạnh nối, mỗi màu là một ca thi.
- **Phân chuồng trong sở thú, phân phòng cho khách** có ràng buộc không ở chung.
- **Cấp phát thanh ghi trong trình biên dịch:** các biến "sống" cùng lúc không được dùng chung một thanh ghi.
- **Phân tần số cho trạm phát sóng:** hai trạm gần nhau phải dùng tần số khác nhau.
- **Giải Sudoku:** Sudoku là bài tô 9 màu trên một đồ thị 81 đỉnh.

Chương 2. Mô tả và mô hình bài toán

2.1. Phát biểu theo kiểu đề thi

Sở thú cần xếp n loài thú vào các chuồng. Có một danh sách các cặp loài "kỵ nhau", tức không được nhốt chung. Sở thú hiện có k chuồng. Câu hỏi đặt ra là có xếp được tất cả các loài vào k chuồng, sao cho không chuồng nào chứa hai loài kỵ nhau, hay không. Nếu có, hãy chỉ ra một cách xếp.

Mở rộng (chế độ "Tìm sắc số χ(G)" của app): không cho trước k. Hỏi cần ít nhất bao nhiêu chuồng, và chỉ ra một cách xếp dùng đúng số chuồng đó.

2.2. Mô hình hóa

Bài toán được đưa về đồ thị vô hướng G = (V, E):

- mỗi loài là một đỉnh;
- mỗi cặp loài kỵ nhau là một cạnh;
- mỗi chuồng là một màu.

Một **cách tô màu hợp lệ** là hàm c: V $\rightarrow$ {0, 1, …, k−1} sao cho với mọi cạnh (u, v) $\in$ E thì c(u) $\neq$ c(v).

Cần phân biệt ba khái niệm:

Khái niệm	Câu hỏi	Trong app
Tô màu hợp lệ	Một cách gán màu cho trước có đúng không?	Do checker kiểm tra
Bài toán k màu (k-coloring)	Có tô được bằng k màu không?	Chế độ "Tô bằng k màu cho sẵn" (mặc định)
Sắc số χ(G)	Số màu nhỏ nhất là bao nhiêu?	Chế độ "Tìm sắc số χ(G)"

Bài toán k màu thuộc lớp NP-đầy đủ khi k ≥ 3 (Karp, 1972). Với k = 2, bài toán giải được trong thời gian đa thức, vì nó tương đương với việc kiểm tra đồ thị có phải đồ thị hai phía hay không.

2.3. Input

Dữ liệu vào là JSON, định nghĩa bằng Zod trong `source-code/src/problems/graph-coloring/definition.ts`:

```
{
  "nodes": ["A1", "B1", "A2", "B2", "A3", "B3"],
  "edges": [{"from": "A1", "to": "B2"}, "..."],
  "colors": 2,
  "mode": "k-coloring"
}
```

- **`nodes`:** danh sách tên đỉnh, mỗi tên dài 1–20 ký tự. **Thứ tự trong mảng chính là thứ tự duyệt** của greedy, quay lui và vét cạn.
- **`edges`:** danh sách cạnh vô hướng, không trọng số.
- **`colors`:** số màu k. Trường này luôn bắt buộc (số nguyên 1–10), nhưng bị bỏ qua ở chế độ χ.
- **`mode`** (không bắt buộc): `"k-coloring"` (mặc định, giao diện ghi "Tô bằng k màu cho sẵn") hoặc `"chromatic"` ("Tìm sắc số χ(G)").

2.4. Output

`{found, coloring, heuristic?, colorsUsed?, chromaticExact?}`, trong đó:

- **`found = true`:** `coloring[i]` là màu của đỉnh thứ i.
- **`found = false` từ thuật toán chính xác:** khẳng định *không tồn tại* cách tô bằng k màu.
- **`found = false` kèm `heuristic: true`:** chỉ có nghĩa là *heuristic chưa tìm được*, không khẳng định vô nghiệm.
- **Ở chế độ χ:** `colorsUsed` là số màu thuật toán đã dùng. Với quay lui và vét cạn, đó chính là χ(G) (`chromaticExact`). Với ba heuristic, đó chỉ là một chặn trên của χ(G).

2.5. Ràng buộc

Ràng buộc	Giá trị
Số đỉnh n (input)	0 ≤ n ≤ 320, cả hai chế độ
Số cạnh m	≤ 8.000
Số màu k	số nguyên 1 ≤ k ≤ 10 (chỉ dùng ở chế độ k

Ràng buộc	Giá trị
	màu)
Greedy, Welsh–Powell, DSATUR	chạy được với mọi input hợp lệ (tới 320 đỉnh). Mức n ≤ 10 chỉ là ngưỡng để ghi trace minh họa, không phải giới hạn chạy
Quay lui	n ≤ 9 ở chế độ k màu, n ≤ 10 ở chế độ χ. Vượt mức thì app báo "Không áp dụng"
Vét cạn	$k^n \le 200.000$ ở chế độ k màu; Σk^n (k = 1..n) ≤ 200.000 ở chế độ χ, tức tối đa 6 đỉnh. Vượt mức thì báo "Không áp dụng". Ngưỡng ghi trace là n ≤ 10; khi quá 5.000 bước hoặc 2,5 MB, trace giữ phần đầu và luôn giữ bước kết quả cuối

Giới hạn n ≤ 9 của quay lui ở chế độ k màu được chọn sau khi đo ca xấu nhất: nhiều đỉnh cô lập đặt trước một clique không tô được làm quay lui phải thử lại mọi tổ hợp màu của các đỉnh cô lập. Ví dụ, với một đồ thị 12 đỉnh ở chế độ k màu, Player hiện: "Không áp dụng: Quay lui chỉ an toàn với đồ thị ≤ 9 đỉnh ở chế độ k-coloring, vì thứ tự đỉnh xấu (ví dụ nhiều đỉnh cô lập trước một clique không tô được) có thể khiến nó thử lại toàn bộ tổ hợp màu của các đỉnh không liên quan (đồ thị hiện có 12 đỉnh)." Số đỉnh ở cuối câu là số đỉnh thật của input.

> ***Ghi chú:*** *ở bản c018f19, chữ "Không áp dụng:" từng bị lặp hai lần trong thông báo của quay lui và vét cạn, vì lý do đã có sẵn tiền tố và Player/So sánh lại thêm một lần nữa. PR #14 (merge commit b0880e1) sửa lỗi này và đã được deploy; câu trích ở trên là nội dung hiển thị trên bản đang chạy.*

2.6. Ví dụ Input/Output

#	Đồ thị	k	Output đúng
1	Tam giác A–B–C	3	found = true, ví dụ A = 0, B = 1, C = 2
2	Tam giác A–B–C	2	found = false: thuật toán chính xác chứng minh vô nghiệm; heuristic báo "chưa

#	Đồ thị	k	Output đúng
			tìm được"
3	Đồ thị vương miện 6 đỉnh (ví dụ chung, Chương 4)	2	found = true, coloring = [0,1,0,1,0,1]. Riêng greedy và Welsh–Powell theo thứ tự xen kẽ thì báo "chưa tìm được"
4	Đồ thị rỗng, n = 0	1	found = true, coloring = []
5	`colors = 0`	–	Bị từ chối vì dữ liệu không hợp lệ

Chương 3. Thiết kế cấu trúc dữ liệu và thuật toán

3.1. Cấu trúc dữ liệu

- **Danh sách đỉnh có thứ tự** (`nodes`): quyết định thứ tự duyệt.
- **Tập cạnh:** checker dùng `Set<string>` chứa các cặp đỉnh có cạnh, nên kiểm tra hai đỉnh có kề nhau hay không trong O(1).
  - **[SV XÁC NHẬN cách lưu kề trong từng `run.ts`]:** có thể là danh sách kề hoặc tập cạnh.
- **Mảng màu** `color[v]`, với giá trị `null` nghĩa là chưa tô.
- **Tập màu bị cấm** của đỉnh v: các màu đang có ở các đỉnh kề với v.
- **Độ bão hòa** sat(v) (chỉ dùng cho DSATUR): số màu *khác nhau* đang có ở các đỉnh kề v.
- **Trace từng bước:** mỗi bước lưu dòng mã giả (`lineId`), trạng thái đồ thị (màu từng đỉnh, đỉnh đang xét), cây trạng thái (`treeNodes`) và lời giải thích. Tham khảo `algorithms/graph-coloring/shared.ts`.

3.2. Phân tích đặc trưng (ý tưởng giải)

1. **Chặn dưới:** nếu đồ thị có một clique (tập đỉnh đôi một kề nhau) kích thước ω, thì cần ít nhất ω màu. Ví dụ, tam giác cần 3 màu.

2. **Chặn trên:** mọi đồ thị tô được bằng Δ + 1 màu, với Δ là bậc lớn nhất. Lý do là khi tô tham lam, mỗi đỉnh có tối đa Δ đỉnh kề nên luôn còn ít nhất một màu trống.

3. **Thứ tự duyệt quyết định chất lượng của tham lam.**

- Luôn tồn tại một thứ tự để greedy cho đúng χ(G) màu.
- Nhưng cũng có thứ tự xấu: trên đồ thị vương miện 2n đỉnh (χ = 2), thứ tự xen kẽ A1, B1, A2, B2… buộc greedy dùng n màu.
- Đây là cơ sở để chọn ví dụ chung.

4. **Ưu tiên đỉnh "khó" trước:** đỉnh bậc cao (Welsh–Powell) hoặc đỉnh bị ràng buộc nhiều nhất (DSATUR). Đỉnh khó mà để sau thì dễ hết màu.

5. **Không gian tìm kiếm của thuật toán chính xác** có k^n cách gán. Quay lui cắt bỏ sớm mọi nhánh vừa tạo xung đột, nhờ vậy duyệt ít hơn nhiều so với vét cạn.

3.3. Năm thuật toán

> *Mã giả dưới đây viết theo dạng sách để phân tích. Mã giả nguyên văn hiển thị trên Player được trích ở mục 4.3.*

(1) Tham lam theo thứ tự đỉnh (Greedy)

```
Greedy(G, order, k)
  for each v in order
    c ← màu nhỏ nhất trong {0..k−1} không có ở đỉnh kề v
    if không có c: return "chưa tìm được"
    color[v] ← c
  return color
```

- **Đúng đắn:** mỗi đỉnh được gán một màu khác với mọi đỉnh kề đã tô. Vì vậy, nếu thuật toán tô được hết thì cách tô hợp lệ.
- **Không bảo đảm** tìm được cách tô dù cách tô tồn tại (phản ví dụ: đồ thị vương miện).
- **Độ phức tạp:**
  - *Lý thuyết* (dùng danh sách kề): O(n·k + m). Mỗi đỉnh quét các đỉnh kề để đánh dấu màu bị cấm, rồi quét k màu.
  - *Cài đặt trong app:* các cạnh được lưu thành tập cặp đỉnh, nên để tìm màu bị cấm, app phải quét toàn bộ n đỉnh. Chi phí là O(n·(n + k)). Muốn đạt O(n·k + m) thì phải chuyển sang danh sách kề.

(2) Welsh–Powell

```
WelshPowell(G, k)
  order ← các đỉnh sắp theo bậc giảm dần (cùng bậc giữ thứ tự ban
đầu)
  return Greedy(G, order, k)
```

- **Ý tưởng:** tô đỉnh nhiều ràng buộc trước. Welsh–Powell bảo đảm số màu tối đa là max_i min(d_i + 1, i), với d_1 ≥ d_2 ≥ … là bậc của các đỉnh sau khi sắp.
- **Độ phức tạp:**
  - *Lý thuyết:* O(n log n + n·k + m).
  - *Cài đặt trong app:* tính bậc bằng cách quét mọi cặp đỉnh, nên tốn $O(n^2)$. Cộng thêm phần sắp xếp và greedy, tổng cộng là $O(n^2 + n \cdot k)$.
- **Lưu ý:** trên đồ thị vương miện, mọi đỉnh cùng bậc 2. Thứ tự sau khi sắp vẫn giữ nguyên thứ tự xen kẽ ban đầu, nên Welsh–Powell gặp đúng khó khăn của greedy (trace thật, mục 4.3).

(3) DSATUR (Brélaz, 1979) – mở rộng ngoài sách, app ghi rõ

```
DSATUR(G, k)
  while còn đỉnh chưa tô
    v ← đỉnh chưa tô có sat(v) lớn nhất; hòa thì chọn bậc lớn hơn
    c ← màu nhỏ nhất không có ở đỉnh kề v
    if không có c: return "chưa tìm được"
    color[v] ← c; cập nhật sat của các đỉnh kề v
  return color
```

- **Tính chất đã biết:** DSATUR luôn tô đúng bằng 2 màu trên đồ thị hai phía.
  - Vì vậy trên ví dụ chung, DSATUR thành công còn greedy thì không. Đây là điểm so sánh chính (trace thật, mục 4.3).
- **Độ phức tạp:**
  - *Lý thuyết:* $O(n^2 + m)$, nếu cập nhật độ bão hòa tăng dần sau mỗi lần tô.
  - *Cài đặt trong app: ở* mỗi vòng, app tính lại độ bão hòa của mọi đỉnh chưa tô bằng cách quét mọi cặp đỉnh. Mỗi vòng tốn $O(n^2)$, n vòng tốn $O(n^3)$. Cộng thêm việc chọn màu $O(n + k)$ mỗi vòng, tổng cộng là $\mathbf{O(n^3 + n \cdot k)}$.

(4) Quay lui (Levitin §12.1)

```
Backtrack(i)
  if i = n: return true                      // đã tô hết
  v ← order[i]
  for c ← 0 to k−1
    if không đỉnh kề nào của v có màu c
      color[v] ← c
      if Backtrack(i+1): return true
      color[v] ← chưa tô                     // quay lui
  return false
```

- **Đúng đắn:**
  - Thuật toán chỉ mở rộng các cách tô bộ phận hợp lệ. Mọi nhánh bị cắt đều đã vi phạm ràng buộc nên không thể mở rộng thành nghiệm.
  - Vì vậy nếu thuật toán trả về false sau khi duyệt hết, thì chắc chắn không có cách tô k màu.
- **Dừng:** *ở* nghiệm đầu tiên, tức là giải bài toán quyết định k màu.
- **Độ phức tạp:**
  - Cây tìm kiếm có tối đa $N = 1 + k + k^2 + \ldots + k^n \le (n + 1) \cdot k^n$ nút, vì mỗi nút có tối đa k con. Ví dụ, khi $k = 1$ thì $N = n + 1$.
  - Mỗi nút kiểm tra một màu với các đỉnh kề. Với danh sách kề, việc này tốn $O(\Delta + 1)$, nên cận lý thuyết là $\mathbf{O((n + 1) \cdot k^n \cdot (\Delta + 1))}$.

- App kiểm tra bằng cách quét mọi đỉnh, mỗi nút tốn O(n), nên trường hợp xấu nhất trong app là **$O(n^2 \cdot k^n)$**.
- Trên thực tế, việc cắt nhánh làm giảm số nút rất nhiều.

(5) Vét cạn

```
Exhaustive(G, k)
  for each phép gán c ∈ {0..k−1}^n (theo thứ tự từ điển)
    if c hợp lệ: return c
  return "vô nghiệm"
```

- **Độ phức tạp:**
  - Có k^n phép gán, tức k^n lá, sinh ra qua tối đa $(n + 1) \cdot k^n$ nút của cây.
  - Nếu kiểm tra mỗi lá bằng cách duyệt danh sách cạnh thì tốn O(m), nên cận là **$O(k^n \cdot (n + m))$**.
  - App kiểm tra bằng cách quét các cặp đỉnh, nên trong app là **$O(k^n \cdot n^2)$**.
  - Đây là mốc để so sánh với quay lui.
- **Giới hạn trong app:**
  - $n \leq 10$ là ngưỡng kích thước input để ghi trace. Input vượt ngưỡng này nhưng vẫn nằm trong giới hạn chạy ($k^n \leq 200.000$ ở chế độ k màu) thì thuật toán vẫn chạy và trả kết quả, nhưng không ghi bước nào; khi đó Player khóa nút Play và giải thích lý do, còn view "Cây trạng thái" hiện thông báo trống.
  - Kể cả khi $n \leq 10$, trace vẫn bị cắt nếu quá 5.000 bước hoặc 2,5 MB. Ví dụ K_6 với k = 5 có $5^6 = 15.625$ phép gán, vô nghiệm, nên sinh ra hơn 5.000 bước. Khi bị cắt, trace giữ phần đầu (vẫn phát được trên Player) và luôn chừa chỗ cho bước kết quả cuối, nên bước cuối vẫn hiện đúng kết quả: cách tô tìm được, hoặc kết luận không thể tô bằng k màu như với K_6, k = 5.
  - Nếu $k^n > 200.000$, app báo "Không áp dụng" thay vì chạy. Ví dụ với 12 đỉnh và k = 3, Player hiện: "Không áp dụng: Vét cạn chỉ an toàn khi $k^n \leq 200.000$ (hiện tại 3^12 = 531.441)."
  - Ở chế độ χ, ngưỡng tính theo Σk^n (k = 1..n): n = 6 cho tổng 67.171, còn n = 7 cho tổng 1.200.304, đã vượt 200.000. Khi đó Player hiện: "Không áp dụng: Vét cạn tìm χ(G) chỉ an toàn khi Σk^n (k=1..n) ≤ 200.000 (đồ thị 7 đỉnh cho tổng 1.200.304)."

Bảng tổng hợp

Thuật toán	Kết luận chắc chắn bài k màu?	Độ phức tạp lý thuyết / trong app	Giới hạn n trong app
Greedy	Không (có thể "chưa tìm được")	$O(n \cdot k + m)$ / $O(n \cdot (n + k))$	320 (ghi trace khi $n \leq 10$)
Welsh–Powell	Không	$O(n \log n + n \cdot k + m)$ / $O(n^2 + n \cdot k)$	320 (ghi trace khi $n \leq 10$)
DSATUR	Không (nhưng đúng trên đồ thị hai phía)	$O(n^2 + m)$ / $O(n^3 + n \cdot k)$	320 (ghi trace khi $n \leq 10$)
Quay lui	Có	$O((n+1) \cdot k^n \cdot (\Delta+1))$ / $O(n^2 \cdot k^n)$	9 (chế độ k màu), 10 (chế độ χ)
Vét cạn	Có	$O(k^n \cdot (n + m))$ / $O(k^n \cdot n^2)$	$k^n \leq 200.000$ (chế độ k màu), $\Sigma k^n \leq 200.000$ tức $n \leq 6$ (chế độ χ); ngưỡng trace $n \leq 10$

Chế độ tìm sắc số χ(G)

Khi chọn "Tìm sắc số χ(G)", tham số k bị bỏ qua:

- **Ba heuristic** tô bằng số màu tùy ý và trả về số màu đã dùng. Con số này chỉ là một **chặn trên** của χ(G).
- **Quay lui** tính hai chặn trước khi tìm:
  - **chặn dưới** là kích thước một clique tìm bằng tham lam (xếp đỉnh theo bậc giảm dần, thêm đỉnh nếu nó kề với mọi đỉnh đã chọn), vì một clique có q đỉnh cần ít nhất q màu;
  - **chặn trên** là số màu DSATUR dùng.

Sau đó nó thử k tăng dần giữa hai chặn. Giá trị k đầu tiên tô được chính là χ(G), vì mọi k nhỏ hơn đã được chứng minh là vô nghiệm.

- **Vét cạn** thử lần lượt k = 1, 2, … và duyệt mọi cách gán với mỗi k, nên tổng số cách gán bị chặn bởi Σk^n.
- **Checker:** với $n \leq 12$, một lời giải tham chiếu độc lập liệt kê các phân hoạch đỉnh thành lớp màu để tính χ(G) và chấm "tối ưu". Số phân hoạch là số Bell

B(n): B(12) ≈ 4,2 triệu còn chạy được, B(15) đã vượt 1,38 tỷ. Với n > 12, checker chỉ xác nhận cách tô hợp lệ, không khẳng định đó là χ(G).

Tài liệu tham chiếu:

- Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed., §1.3 và §12.1;
- Welsh & Powell (1967);
- Brélaz (1979).

Mã giả trong report được viết lại từ mã giả hiển thị trong app. Mã giả trong app do công cụ AI sinh ra (xem Chương 5).

Chương 4. Ví dụ minh họa các bước thực hiện

4.1. Ví dụ chung

Ví dụ chung là đồ thị vương miện 6 đỉnh, với k = 2:

- **Đỉnh,** theo thứ tự nhập: A1, B1, A2, B2, A3, B3.
- **Cạnh:** Ai–Bj với i ≠ j, gồm A1–B2, A1–B3, A2–B1, A2–B3, A3–B1, A3–B2.
- **Ý nghĩa:** loài Ai kỵ mọi loài Bj trừ Bi. Đồ thị là hai phía (nhóm A, nhóm B), nên χ = 2. Tuy vậy thứ tự xen kẽ là thứ tự "xấu" với greedy.

[ẢNH: đồ thị trên Player trước khi chạy]

Lý do chọn ví dụ này:

- đủ nhỏ để chạy tay;
- có một lần quay lui thật;
- các thuật toán cho kết quả khác nhau;
- vét cạn vẫn chạy được, vì $2^6 = 64$ cách gán.

JSON để tái hiện: xem `docs/kich-ban-video.md` mục 5.1.

4.2. Quay lui trên ví dụ chung (khớp đúng trace của Player)

Bảng dưới đây lấy từ lần chạy thật hàm `run()` của app, với cùng bộ ghi trace mà Player dùng. Lần chạy được lặp hai lần và cho kết quả giống nhau.

- Player đánh số bước từ 1. Bước 0 là trạng thái ban đầu.
- Cột "Mảng màu" theo thứ tự A1 B1 A2 B2 A3 B3; "–" là chưa tô.
- Cột "Cây" là nút tương ứng trên cây trạng thái.

Bước	Dòng mã giả	Giải thích hiển thị trên Player	Mảng màu	Cây
1	init	Tô 6 đỉnh theo thứ tự cho sẵn.	– – – – – –	gốc
2	place	Gán màu 0 cho đỉnh A1.	0 – – – – –	A1:0
3	place	Gán màu 0 cho đỉnh B1.	0 0 – – – –	B1:0
4	reject	Màu 0 trùng với đỉnh kề B1 đã tô	0 0 – – – –	A2:0 ✗

Bước	Dòng mã giả	Giải thích hiển thị trên Player	Mảng màu	Cây
		— loại bỏ.		
5	place	Gán màu 1 cho đỉnh A2.	0 0 1 – – –	A2:1
6	reject	Màu 0 trùng với đỉnh kề A1 đã tô — loại bỏ.	0 0 1 – – –	B2:0 ✗
7	place	Gán màu 1 cho đỉnh B2.	0 0 1 1 – –	B2:1
8	reject	Màu 0 trùng với đỉnh kề B1 đã tô — loại bỏ.	0 0 1 1 – –	A3:0 ✗
9	reject	Màu 1 trùng với đỉnh kề B2 đã tô — loại bỏ.	0 0 1 1 – –	A3:1 ✗
10	backtrack	Không màu nào bên dưới đỉnh B2 khả thi — quay lui.	0 0 1 – – –	về B2:1
11	backtrack	Không màu nào bên dưới đỉnh A2 khả thi — quay lui.	0 0 – – – –	về A2:1
12	backtrack	Không màu nào bên dưới đỉnh B1 khả thi — quay lui.	0 – – – – –	về B1:0
13	place	Gán màu 1 cho đỉnh B1.	0 1 – – – –	B1:1
14	place	Gán màu 0 cho đỉnh A2.	0 1 0 – – –	A2:0
15	reject	Màu 0 trùng với đỉnh kề A1 đã tô — loại bỏ.	0 1 0 – – –	B2:0 ✗
16	place	Gán màu 1 cho	0 1 0 1 – –	B2:1

Bước	Dòng mã giả	Giải thích hiển thị trên Player	Mảng màu	Cây
		đỉnh B2.		
17	place	Gán màu 0 cho đỉnh A3.	0 1 0 1 0 –	A3:0
18	reject	Màu 0 trùng với đỉnh kề A1 đã tô — loại bỏ.	0 1 0 1 0 –	B3:0 ✗
19	place	Gán màu 1 cho đỉnh B3.	0 1 0 1 0 1	B3:1
20	done	Tô thành công với số màu đã cho.	0 1 0 1 0 1	—

Phân tích:

- **Bước 8–9:** A3 kề cả B1 (màu 0) lẫn B2 (màu 1), nên với k = 2 thì không còn màu nào. Đây là điểm bế tắc.
- **Bước 10–12:** thuật toán quay lui qua B2 và A2, vì hai đỉnh này không còn màu nào chưa thử. Nó dừng lại ở B1.
  - Gốc của bế tắc là quyết định ở bước 3: B1 lấy màu 0, trùng màu với A1.
  - Hai đỉnh này không kề nhau nên gán như vậy là hợp lệ. Nhưng nó buộc mọi đỉnh A còn lại phải khác màu B1, và mọi đỉnh B còn lại phải khác màu A1. Với hai màu, điều này dẫn tới bế tắc ở A3.
- **Bước 13:** B1 đổi sang màu 1. Từ đây nhóm A nhận màu 0 và nhóm B nhận màu 1, và không còn bế tắc nào nữa.
- **Thống kê:** 20 bước, 15 lần thăm nút, 6 lần cắt nhánh (reject), tổng 21 phép toán.
  - Cây trạng thái có 15 nút thử màu: 9 lần gán (`place`) và 6 lần loại (`reject`). Vét cạn duyệt theo thứ tự từ điển và gặp nghiệm đầu tiên 010101 ở lá thứ 22. Không gian tìm kiếm của vét cạn có tối đa $2^6 = 64$ lá.

Kết quả: found = true, coloring = [0, 1, 0, 1, 0, 1]. Checker kiểm tra cả 6 cạnh, hai đầu mút của mỗi cạnh đều khác màu.

> ***Lưu ý:*** *sau PR #12, trace này vẫn giữ đúng 20 bước. Dòng* `try` *cũ đã bị bỏ, nên việc thử màu hiện trên dòng* `reject` *(bước loại) hoặc* `place` *(bước gán).*

[ẢNH: Player ở bước 9 (bế tắc) và bước 12 (quay lui về B1), kèm cây trạng thái]

[ẢNH: Player ở bước 20]

[ẢNH: bản chạy tay trên giấy của sinh viên – SV XÁC NHẬN]

4.3. Cùng ví dụ, bốn thuật toán còn lại (trace thật)

Thuật toán	Số bước	Phép toán	Kết quả	Checker
Greedy	7	5	Tô A1 = 0, B1 = 0, A2 = 1, B2 = 1; tới A3 thì hết màu → "chưa tìm được"	Chấp nhận (heuristic), không kết luận vô nghiệm
Welsh–Powell	7	5	Mọi đỉnh cùng bậc 2 nên thứ tự giữ nguyên → giống greedy	Như trên
DSATUR	14	6	A1 = 0 → B2 = 1 → A3 = 0 → B1 = 1 → A2 = 0 → B3 = 1: **tô thành công**	Hợp lệ
Quay lui	20	21	[0,1,0,1,0,1]	Hợp lệ
Vét cạn	69	67	Nghiệm đầu tiên theo thứ tự từ điển, gặp ở bước 68	Hợp lệ

Nhận xét:

- **Vì sao DSATUR thành công:**
  - Sau khi tô A1, các đỉnh kề A1 (B2, B3) có độ bão hòa 1, nên được chọn trước B1.
  - B2 nhận màu 1, rồi A3 (kề B2) nhận màu 0, và cứ thế lan ra.
  - Thuật toán luôn tô đỉnh "bị ép" nhiều nhất, nên không bao giờ rơi vào cách gán B1 = 0 như greedy.
  - Kết quả này đúng với tính chất đã biết: DSATUR tô đúng mọi đồ thị hai phía.
- **Greedy và Welsh–Powell** thất bại vì thứ tự đỉnh. Đây là đúng phản ví dụ ở §3.2.

- **Quay lui** dùng 20 bước và 21 phép toán, còn vét cạn dùng 69 bước và 67 phép toán. Phần chênh lệch là nhờ cắt nhánh sớm.

Cùng ví dụ ở chế độ "Tìm sắc số χ(G)" (chạy `run()` của app hai lần, kết quả giống nhau):

Thuật toán	Số màu dùng	Số bước	Phép toán	Kết quả
Greedy	3	8	6	Hợp lệ nhưng chưa tối ưu
Welsh–Powell	3	8	6	Hợp lệ nhưng chưa tối ưu
DSATUR	2	14	6	Tối ưu
Quay lui	2	22	21	χ = 2, chính xác
Vét cạn	2	78	74	χ = 2, chính xác

Ở chế độ này greedy không còn bị giới hạn k = 2. Nó mở thêm màu thứ ba khi tới A3, nên tô được nhưng dùng 3 màu, trong khi χ = 2. Đây là cách nhìn khác của cùng một phản ví dụ: với thứ tự đỉnh xấu, greedy có thể dùng nhiều màu hơn mức cần.

Mã giả của từng thuật toán trên Player, chép nguyên văn từ `algorithms/graph-coloring/<thuật toán>/pseudocode.ts`:

Greedy

1. Xét các đỉnh theo thứ tự cho trước
2. Gán màu nhỏ nhất trong k màu chưa dùng ở các đỉnh kề đã tô
3. Nếu không còn màu nào khả dụng: dừng — heuristic chưa tìm được, không chứng minh vô nghiệm
4. Trả về cách tô nếu tô được hết mọi đỉnh

Welsh–Powell

1. Sắp các đỉnh theo bậc giảm dần
2. Gán màu nhỏ nhất trong k màu chưa dùng ở các đỉnh kề đã tô
3. Nếu không còn màu nào khả dụng: dừng — heuristic chưa tìm được, không chứng minh vô nghiệm

4. Trả về cách tô nếu tô được hết mọi đỉnh

DSATUR

1. Khởi tạo độ bão hoà (số màu khác nhau ở các đỉnh kề) bằng 0
2. Chọn đỉnh chưa tô có độ bão hoà lớn nhất, hoà thì chọn bậc lớn hơn
3. Gán màu nhỏ nhất trong k màu chưa dùng ở các đỉnh kề đã tô
4. Nếu không còn màu nào khả dụng: dừng — heuristic chưa tìm được, không chứng minh vô nghiệm
5. Trả về cách tô nếu tô được hết mọi đỉnh

Quay lui

1. Tô đỉnh theo thứ tự 0..n−1
2. Thử màu c cho đỉnh hiện tại; nếu trùng màu một đỉnh kề đã tô: loại bỏ c
3. Nếu c hợp lệ: gán màu, sang đỉnh kế tiếp
4. Nếu hết k màu mà không màu nào hợp lệ: quay lui
5. Trả về cách tô nếu tô được hết n đỉnh, hoặc báo vô nghiệm
6. Chế độ χ(G): tìm một clique bằng tham lam để có chặn dưới
7. Chế độ χ(G): tô bằng DSATUR để có chặn trên
8. Chế độ χ(G): tăng k từ chặn dưới, quay lui kiểm tra khả thi với từng k
9. Chế độ χ(G): k nhỏ nhất khả thi chính là χ(G)

Vét cạn

1. Sinh mọi cách tô (mỗi đỉnh chọn một trong k màu, k^n cách)
2. Gán màu cho đỉnh hiện tại
3. Khi đã gán hết n đỉnh: kiểm tra toàn bộ đồ thị — nếu không có hai đỉnh kề cùng màu, đây là nghiệm
4. Trả về nghiệm đầu tiên tìm được, hoặc báo vô nghiệm
5. Chế độ χ(G): thử k = 1, 2, … và vét cạn lại với mỗi k
6. Chế độ χ(G): k nhỏ nhất khả thi chính là χ(G)

> ***Ghi chú:*** *bản trước PR #12 có hai chỗ lệch: dòng "báo không thể với k màu" của heuristic mâu thuẫn với lời giải thích "chưa tìm được", và dòng* `try` *của quay lui không bao giờ được tô sáng. PR #12 đã sửa cả hai: heuristic ghi rõ "chưa tìm được,*

> *không chứng minh vô nghiệm", còn bước thử màu được gộp vào dòng `reject`. Số bước ở chế độ k màu không đổi (7/7/14/20/69).*

[ẢNH: trang So sánh với ví dụ chung]

Chương 5. Tường thuật quá trình thiết kế và xây dựng

5.1. Nói rõ ngay từ đầu

Toàn bộ mã nguồn của hệ thống được sinh bằng các công cụ AI, gồm Claude Code (Anthropic) và Codex (OpenAI). Việc giao việc, review chéo và kiểm tra được thực hiện qua một agent điều phối (orchestrator) chạy trên nền tảng Omnigent.

Những việc em trực tiếp làm, có lưu vết trong lịch sử làm việc:

- đặt yêu cầu và phạm vi, bao gồm quyết định làm toàn bộ sách thay vì một bài;
- chọn công nghệ và phong cách giao diện (shadcn/ui, tông trắng đen);
- chọn để repo private và host trên Cloudflare Pages, gắn domain riêng `hieutk.dev`;
- yêu cầu các mô hình tranh luận về kiến trúc và về chiến lược làm report;
- merge PR #2 đến #8 vào nhánh chính sau khi review chéo đạt. GitHub ghi `mergedBy: tkhieu`;
- xử lý các lần agent bị chặn: duyệt ngân sách, đăng nhập lại khi hết phiên, mở khóa keyring;
- **[CHƯA LÀM – SV phải làm và xác nhận trước khi nộp; xoá dòng này nếu không làm]** tự kiểm chứng thuật toán bằng chạy tay (mục 5.6).

Em **không** tự viết code và **không** tự review code. Việc review là do một agent của hãng khác thực hiện, và em đọc kết luận review để quyết định có merge hay không.

5.2. Công cụ sử dụng

Công cụ	Vai trò
Claude Code (model Claude Sonnet / Opus) [SV XÁC NHẬN phiên bản]	Viết code các nhóm thuật toán, review code do Codex viết
Codex (model GPT) [SV XÁC NHẬN phiên bản]	Viết code, review code do Claude Code viết
Grok (qua Cursor)	Tham gia tranh luận chiến lược, không viết code
Omnigent (agent điều phối "jacky")	Chia việc, tạo worktree, chạy lại các bước kiểm tra, dựng bản diff cố định để review
Vite, React, TypeScript, shadcn/ui, Zustand,	Công nghệ của app

Công cụ	Vai trò
Zod, Vitest, Playwright	
GitHub (repo private), Cloudflare Pages	Quản lý mã nguồn, deploy

Nguyên tắc review chéo khác hãng: việc do Claude Code làm thì Codex review, và ngược lại. Reviewer chỉ nhận bản diff cố định theo commit, không được sửa code. Lỗi chặn được gửi lại cho bên làm để sửa, kèm yêu cầu có test hồi quy fail trên code cũ.

5.3. Dòng thời gian

Giai đoạn	Nội dung	Kết quả
1. Phân tích đề	AI tách đề thành note có cấu trúc (`docs/de-bai-bai-tap-lon.md`)	
2. Khảo sát sách	Hai agent đọc song song sách Levitin, lập danh sách khoảng 60 thuật toán demo được (`docs/levitin-thuat-toan-co-the-demo.md`)	
3. Thiết kế kiến trúc	Claude và Codex mỗi bên đưa một đề xuất, rồi phản biện chéo và chốt (`docs/kien-truc-ung-dung-demo.md`)	
4. Bản đầu: cái túi 0/1	Codex làm, Claude review	PR #1, merge 11/10
5. Deploy	GitHub Pages bị từ chối với repo private, chuyển sang Cloudflare Pages	
6. Nền tảng catalog	Claude bắt đầu, hết phiên đăng nhập; Codex làm nốt	PR #2
7. Năm nhóm song song	B1–B5, mỗi nhóm có worktree và PR riêng	PR #3–#7
8. Tích hợp	Kiểm tra độ phủ, e2e đi qua	PR #8, merge 12/10

Giai đoạn	Nội dung	Kết quả
	mọi trang, xuất testcase	
9. Domain riêng	algorithm-visualizer.hieutk.dev	17/10
10. Chiến lược report	Ba mô hình tranh luận, chọn tô màu làm trọng tâm	18/10
11. Nâng cấp tô màu	Chế độ χ(G), heuristic chạy tới 320 đỉnh, bộ sinh đồ thị, 22 testcase. Claude Code làm, Codex review; sau 3 vòng CHANGES REQUESTED, vòng sửa cuối do Codex làm và Claude Code review PASS	PR #12, merge 18/10

Bảng chi tiết 8 PR (merge commit, số vòng review) nằm ở Phụ lục A.

5.4. Thiết kế chức năng và giao diện

- **Prompt giao diện em dùng (nguyên văn, lấy từ lịch sử phiên điều phối, giữ nguyên chính tả gốc):**

> *"Hãy chọn 1 đề tài mà bạn nghĩ là hợp lý,. build theo kiến trúc khuyến nghhị, build cho tới khi xong có thể demo đựoc. UI chọn shadcn ui tông màu sáng. trắng đen."*

- **Cách hiện thực:**
  - Đặc tả yêu cầu: vì chỉ dùng trắng, đen và xám, các trạng thái (đang xét, đã chọn, bị cắt, tốt nhất) phải được phân biệt bằng độ đậm và kiểu viền, không dựa vào màu. Cách này cũng giúp người mù màu đọc được.
  - Với bài tô màu, khung đồ thị hiển thị màu của từng đỉnh bằng số trên nhãn và làm nổi bật đỉnh đang xét. Cây trạng thái đánh dấu riêng các nhánh bị cắt. Player hiện **chưa** hiển thị bảng chú thích (legend) cho bài này.
- **Các chức năng chính:**
  - Player từng bước, gồm mã giả có dòng đang chạy, lời giải thích tiếng Việt, tiến/lùi/tua và phím tắt;
  - trang So sánh nhiều thuật toán trên cùng input;
  - trang chạy Testcase, có xuất CSV/JSON;

    - Catalog với điều hướng theo chương, theo kỹ thuật, theo bài toán, có tìm kiếm `Ctrl+K`.
- **Kiến trúc:** thuật toán chạy trong Web Worker để không làm đứng giao diện. Mỗi thuật toán ghi trace qua một đối tượng `Recorder` dùng chung. Bản ghi đủ để minh họa từng bước, còn bản chỉ đếm số phép toán dùng để đo thời gian công bằng. Như vậy không phải viết hai phiên bản của cùng một thuật toán.
- **Mở rộng sang cả sách**, prompt nguyên văn:

> *"Dựa theo phần đã làm hãy build theo kiến trúc khuyến nghhị, build cho tới khi xong có thể demo đượoc. – Tất cả các bài toán và cách giải – Navigation cần được build lại để có thể access tới các bài toán, hoặc access tới các cách giải."*

[ẢNH: Player, trang So sánh, trang Testcase]

5.5. Các sự cố tiêu biểu và cách xử lý

> *Prompt dưới đây được trích nguyên văn từ log phiên trong `~/.claude/projects/…HomeWork/*.jsonl`, hoặc từ lịch sử phiên điều phối. Chỗ nào chỉ là bản tái dựng thì ghi rõ.*

Sự cố 1 – Bài tô màu đồ thị bị review FAIL với 4 lỗi (PR #7)

- **Mục tiêu:** cài 5 thuật toán tô màu cho nhóm B5.
- **Công cụ:** Claude Code viết code, Codex review, rồi Codex sửa và Claude Code review lại.
- **Prompt giao việc** (nguyên văn, trích đoạn đầu, log `c6dcc10b-…jsonl`):

> *"/goal IMPLEMENT group B5 (tsp, assignment, n-queens, hamiltonian-circuit, subset-sum, graph-coloring (bai So thu), matrix-chain, lcs…"*

- **Kết quả lượt đầu:** code chạy, mọi test pass, nhưng review cross-vendor kết luận **FAIL**. Bốn lỗi liên quan tới bài tô màu:

    1. **Checker tự tham chiếu:** checker kiểm "vô nghiệm" bằng chính kiểu quay lui của thuật toán đang được kiểm. Nếu thuật toán sai thì checker cũng sai theo, và test vẫn pass.
    2. **Heuristic khẳng định sai:** khi không tô được, heuristic báo "không thể tô", trong khi thực ra nó chỉ chưa tìm được.
    3. **Đồ thị rỗng (n = 0):** quay lui và vét cạn chưa xử lý trường hợp này.

4. **Hình không cập nhật khi quay lui:** khung đồ thị trên Player không đổi màu lại.

- **Cách sửa (commit 4b83502):**
  - Viết oracle độc lập `existsColoring`: liệt kê các cách chia đỉnh thành các tập độc lập, không dùng phép gán màu kiểu quay lui.
  - Thêm cờ `heuristic`. Khi cờ này bật, checker chấp nhận kết quả "chưa tìm được" nhưng không coi đó là tối ưu.
  - Thêm test hồi quy `review-regressions.test.ts`, trong đó có chính đồ thị vương miện của ví dụ chung.
- **Prompt review lượt 2** (nguyên văn, log `04cbfb0d-…jsonl`):

  > *"REVIEW (cross-vendor; fixes by Codex, original by Claude Code) of catalog group B5… Previous review FAILED with 8 blocking issues…"*

- **Bài học:** "test pass" chưa chứng minh được là code đúng. Lời giải tham chiếu phải độc lập với thuật toán đang kiểm. Ngoài ra, một heuristic không bao giờ được khẳng định là vô nghiệm.

Sự cố 2 – Nhánh cận đánh dấu sai nút tốt nhất (PR #1, bài cái túi)

- **Lỗi:** đúng lúc tìm được nghiệm tốt nhất mới, nút đó lại bị đánh dấu là "bị cắt nhánh", kèm lời giải thích sai.
  - Kết quả cuối vẫn đúng, nên test cũ không phát hiện được.
  - Nhưng phần minh họa sai đúng ở khoảnh khắc quan trọng nhất.
- **Ai phát hiện:** Claude Code, khi review chéo phần code do Codex viết.
- **Sửa:** commit c0e1741. Test hồi quy `bestNeverPruned` được thêm ở commit 7857a79. Có 8 test hồi quy, và cả 8 **fail trên code cũ** 5fde3ef. Orchestrator đã chạy lại riêng các test này trên commit cũ để xác nhận.
- **Bài học:** với một app minh họa, *trace sai cũng là lỗi*, dù đáp án cuối vẫn đúng.

Sự cố 3 – Lời giải tham chiếu tự tham chiếu lặp lại ở nhiều nhóm

- **Lỗi:** ở các nhóm B1, B4, B5, nhiều test so kết quả với một "lời giải tham chiếu" viết lại y hệt chính thuật toán. Ví dụ, phần tử nhỏ thứ k được so với sắp xếp rồi chọn, và công thức Josephus được so với chính công thức đó.
- **Sửa:** thay bằng tham chiếu độc lập, ví dụ:
  - đếm số phần tử nhỏ hơn đáp án;

    - mô phỏng trực tiếp Josephus;
    - Held-Karp cho TSP;
    - monotone chain cho bao lồi.
- **Kết quả:** quy tắc "test phải so với lời giải tham chiếu độc lập" được ghi vào `source-code/docs/adding-an-algorithm.md` và `AGENTS.md`.

Sự cố 4 – Code viết dồn một dòng (PR #3, #4)

- **Lỗi:** 136 trên 184 file mới bị viết dồn thành một dòng, có dòng dài hơn 1.000 ký tự. Code vẫn chạy và qua lint, nhưng không đọc được.
- **Lần sửa đầu gây lỗi mới:** agent định dạng lại luôn cả file dùng chung `manifest.ts`, khoảng 1.650 dòng thay đổi. Như vậy nhóm nào merge sau cũng sẽ xung đột.
- **Sửa lần hai:** trả các file dùng chung về cách viết cũ, chỉ giữ đúng các dòng cần thay đổi.
- **Bài học:** chỉ ghi "phải chạy được" là chưa đủ. Yêu cầu giao cho AI phải nêu cả tiêu chí dễ đọc và phạm vi file được sửa.

(Tuỳ chọn) Sự cố 5 – Trang So sánh cần 6 vòng review (PR #8)

Mỗi vòng sửa lại lộ thêm một trường hợp biên khi người dùng đổi lựa chọn thuật toán hoặc rời trang trong lúc đang chạy. Các commit sửa là 8698427, 0069f32, aaab204, e3908f8. Sự cố này cho thấy giới hạn của review từng vòng. Từ đó, orchestrator yêu cầu reviewer "nêu hết lỗi trong một lần".

5.6. Phần sinh viên tự kiểm chứng

> ***[SV XÁC NHẬN – bắt buộc phải tự làm]***

- Chạy tay quay lui trên ví dụ chung (bảng ở mục 4.2), chụp ảnh bản viết tay, rồi đối chiếu từng bước với Player.
- Giải tay 3 testcase nhỏ (tam giác với k = 2, đồ thị đầy đủ K4 với k = 4, đồ thị rỗng), rồi so với kết quả của app.
- Tự kiểm tra độ phức tạp đã ghi ở Chương 3 bằng cách đếm số nút của cây quay lui trên ví dụ chung.
- Ghi lại những chỗ em không đồng ý hoặc phải hỏi lại AI.

5.7. Đánh giá việc dùng AI

- **AI làm tốt:** dựng nhanh một hệ thống lớn (47 bài toán, 126 cách giải trong khoảng 22 giờ), giữ đúng kiến trúc chung, tự viết test.
- **AI hay sai:**
  - test tự tham chiếu;
  - trace minh họa sai dù đáp án đúng;
  - "làm cho có", ví dụ simplex giả hay Boyer-Moore thiếu bảng hậu tố tốt (PR #6);
  - code khó đọc.
- **Review chéo khác hãng bắt được:** tất cả các lỗi trên đều do reviewer của hãng còn lại phát hiện, không phải do bên viết code tự phát hiện.
- **Vẫn phải do người kiểm:**
  - phạm vi và tính trung thực của báo cáo;
  - việc chạy tay để hiểu thuật toán;
  - quyết định merge.
- **Giới hạn của oracle hiện tại:** liệt kê phân hoạch chỉ khả thi với đồ thị nhỏ ($n \leq 12$). Với đồ thị lớn hơn, checker chỉ kiểm được cách tô có hợp lệ hay không, không biết được $\chi(G)$.

Chương 6. Thiết kế thực nghiệm

6.1. Cách tạo testcase

- **Viết tay:** các ca nhỏ có đáp án dễ thấy.
- **Dựng có chủ đích:** các họ đồ thị đã biết χ. Trên giao diện, bộ sinh mẫu ghi: "Vương miện (2 nhóm kích thước m, χ = 2)", "Đầy đủ K_n (χ = n)", "Chu trình C_n (χ = 2 nếu n chẵn, 3 nếu n lẻ)", "Hai phía đầy đủ K(p,q) (χ = 2)", "Petersen (10 đỉnh, χ = 3)", "Grötzsch — Mycielski của C5, không tam giác (11 đỉnh, χ = 4)". Tham số được kiểm tra trước khi dựng; vượt giới hạn thì app từ chối, không cắt bớt cạnh.
- **Sinh ngẫu nhiên theo seed** với n (1–320) và xác suất cạnh p (đưa về [0, 1]). Cùng seed luôn cho cùng đồ thị. App tự hạ p để số cạnh kỳ vọng không quá 7.200, và từ chối nếu số cạnh thực vượt 8.000.
- **Xuất file:** testcase được xuất bằng lệnh export của app ra `testcases/graph-coloring/`.

6.2. Phân nhóm

Có 22 file trong `testcases/graph-coloring/`. Cột "Kỳ vọng" là χ đã biết; "—" là ca không khai báo kỳ vọng, khi đó checker tự kiểm (cách tô hợp lệ, hoặc oracle độc lập với ca vô nghiệm).

Nhóm	File	n / m	Chế độ, k	Kỳ vọng	Mục đích
Thông thường	normal-1, normal-2	3/3	k màu, k = 3 và 2	—	Tam giác, có nghiệm / vô nghiệm
Thông thường	chromatic-cycle-odd, chromatic-cycle-even	5/5, 6/6	χ	3, 2	Chu trình lẻ cần 3 màu, chẵn cần 2
Thông thường	chromatic-bipartite	7/12	χ	2	Đồ thị hai phía
Biên	edge-single, edge-no-edges	1/0, 3/0	k màu, k = 1	—	Một đỉnh; không có cạnh
Biên	chromatic-	0/0	χ	0	Đồ thị rỗng

Nhóm	File	n / m	Chế độ, k	Kỳ vọng	Mục đích
	empty				
Nhỏ	small-1, small-2	5/3, 5/4	k màu, k = 3 và 4	—	
Lớn	large-1, large-2	7/10, 7/9	k màu, k = 2 và 1	—	
Lớn	chromatic-large-100, -200, -300	100/166, 200/409, 310/709	χ	—	Ca lớn sinh theo seed. Chỉ heuristic chạy được; trang Testcase tự chuyển sang heuristic khi thuật toán tham chiếu báo "Không áp dụng"
Đặc biệt	special-complete4	4/6	k màu, k = 4	—	K_4 cần đúng 4 màu
Đặc biệt	chromatic-crown	6/6	χ	2	Ví dụ chung, bẫy của greedy
Đặc biệt	chromatic-complete-k6	6/15	χ	6	K_6
Đặc biệt	chromatic-petersen, chromatic-grotzsch	10/15, 11/20	χ	3, 4	Đồ thị kinh điển; Grötzsch không có tam giác mà vẫn cần 4 màu
Không hợp lệ	invalid-zero-colors, invalid-mode	1/0	k = 0; mode sai	bị từ chối	Kiểm tra dữ liệu vào

Lưu ý: ca "chromatic-large-300" thực tế có 310 đỉnh.

6.3. Đo đạc

- **Số đo:** đúng/sai theo checker, số bước, số phép toán, thời gian.

- **Cách đo thời gian:** trang So sánh đo nhiều lần và lấy trung vị, ở chế độ chỉ đếm, không ghi trace.
- **Cấu hình máy đo:** số liệu ở §7.2 đo bằng Node v24.21.0 trên Intel Core Ultra 7 270K Plus (24 CPU logic), gọi thẳng `run()` của app với bộ đếm chỉ tính metrics, không ghi trace. Nếu chụp thêm số liệu từ trang So sánh trên trình duyệt thì ghi rõ [SV ĐIỀN: trình duyệt, phiên bản, RAM].
- **Ca vượt giới hạn:** ghi "Không áp dụng", không đưa vào biểu đồ.

6.4. Bảo đảm độ phủ

Mỗi nhánh của thuật toán phải có ít nhất một testcase đi qua:

Nhánh cần phủ	Testcase
Có nghiệm	normal-1
Vô nghiệm, thuật toán chính xác chứng minh	normal-2, large-2
Heuristic "chưa tìm được" dù có nghiệm	đồ thị vương miện ở chế độ k màu, k = 2 (unit test của greedy; chưa có file testcase riêng)
Heuristic tô được nhưng chưa tối ưu	chromatic-crown ở chế độ χ: greedy trả về cách tô hợp lệ với 3 màu, trong khi χ = 2
Đồ thị rỗng hoặc không có cạnh	edge-no-edges, ca n = 0 trong unit test
Đồ thị đầy đủ	special-complete4
Dữ liệu sai	invalid-zero-colors, invalid-mode
Chế độ χ, có χ đã biết	chromatic-cycle-*, chromatic-bipartite, chromatic-complete-k6, chromatic-petersen, chromatic-grotzsch
Vượt giới hạn của thuật toán chính xác	chromatic-large-100/200/300 (quay lui và vét cạn báo "Không áp dụng")

Chương 7. Đánh giá

7.1. Tính đúng

- **Trang Testcase:** "Đạt 22/22", không ca nào "Không áp dụng". Bằng chứng gồm hai lần chạy riêng: (1) bộ e2e chạy local trên dev server lúc merge PR #12; (2) ngày 18/10, cùng bộ e2e được chạy lại với một cấu hình Playwright tạm có `baseURL` trỏ thẳng tới https://algorithm-visualizer.hieutk.dev, không bật dev server, và pass 314/314, trong đó có kiểm tra "Đạt 22/22" của trang Testcase. Sau khi deploy bản sửa PR #14 (b0880e1), lần chạy lại cùng cấu hình đó cũng pass 314/314. [ẢNH: trang Testcase]
- **Unit test:**
  - Quay lui được so với oracle độc lập `existsColoring`.
  - Vét cạn được so với quay lui.
  - Có test hồi quy riêng cho các lỗi đã gặp.
- **Toàn app:** lúc merge PR #12 có 825 unit test và 314 e2e đều pass (cộng 2 e2e chạy dưới đường dẫn con), trong đó có e2e tự đi qua mọi trang Player, So sánh và Testcase.
  - **Ý nghĩa và giới hạn:** các con số này cho thấy mọi trang chạy được với input mặc định. Chúng không thay thế việc chứng minh đúng đắn ở Chương 3.

7.2. Kết quả so sánh

Trên ví dụ chung: xem bảng ở §4.3. Ở chế độ χ, greedy và Welsh–Powell dùng 3 màu, còn DSATUR, quay lui và vét cạn dùng đúng $\chi = 2$. Quay lui cần 22 bước, vét cạn 78 bước.

Trên đồ thị ngẫu nhiên lớn G(n, p = 0,1), 5 seed (1–5), lấy trung vị. Mỗi ô ghi thời gian (ms) / số màu dùng:

n	Greedy	Welsh–Powell	DSATUR
50	0,098 / 5	0,241 / 4	1,173 / 4
100	0,279 / 7	0,692 / 6	7,958 / 6
200	1,359 / 12	3,181 / 10	64,816 / 8
300	3,579 / 14	7,363 / 13	203,489 / 11

n	Greedy	Welsh–Powell	DSATUR

[ẢNH: trang So sánh trên ví dụ chung và trên một ca lớn]

Nhận xét:

- **Chất lượng:** ở mọi kích thước, DSATUR dùng ít màu nhất, rồi tới Welsh–Powell, cuối cùng là greedy. Ở n = 300, DSATUR dùng 11 màu so với 14 của greedy.
- **Thời gian:** greedy nhanh nhất. DSATUR chậm hơn rõ rệt và tăng nhanh theo n: từ n = 100 lên n = 200, thời gian tăng khoảng 8 lần, khớp với độ phức tạp $O(n^3)$ trong app ở §3.3 (gấp đôi n thì gấp 8 lần). Dù vậy 300 đỉnh vẫn chỉ mất khoảng 0,2 giây.
- **Không biết χ thật:** với các đồ thị trên 12 đỉnh này app không tính được χ(G), nên chỉ so được các heuristic với nhau, không khẳng định được cái nào tối ưu.
- **Quay lui:** chế độ χ trên đồ thị ngẫu nhiên thưa n = 8, 9, 10 chạy rất nhanh (khoảng 0,04 ms, đều ra 2 màu). Đây là trường hợp dễ. Giới hạn n ≤ 10 được chọn theo ca xấu nhất, không theo ca trung bình (xem §2.5).

7.3. Tính trực quan và khả năng minh họa

- Mỗi bước đều có dòng mã giả, lời giải thích tiếng Việt và trạng thái đồ thị. Có thể lùi lại bất kỳ bước nào.
- Cây trạng thái cho thấy rõ lúc nào quay lui.
- Giao diện trắng đen vẫn đọc được trạng thái nhờ số màu trên nhãn đỉnh, đỉnh đang xét được làm nổi bật, và các nhánh bị cắt trên cây. Hạn chế: chưa có bảng chú thích (legend) trên Player.

7.4. Hạn chế

- Thuật toán chính xác chỉ chạy với đồ thị nhỏ: quay lui tới 9 đỉnh (chế độ k màu) hoặc 10 đỉnh (chế độ χ), vét cạn tới $k^n \le 200.000$ hoặc 6 đỉnh. Giới hạn của quay lui chọn theo ca xấu nhất khi giữ nguyên thứ tự đỉnh như bài giảng, nên thấp hơn khả năng của một quay lui có sắp xếp đỉnh.

- Với đồ thị trên 12 đỉnh, app không biết χ(G) thật, nên không chấm được heuristic có tối ưu hay không.
- Trường `colors` vẫn bắt buộc ở chế độ χ dù bị bỏ qua.
- Trace minh họa chỉ ghi đủ với đồ thị nhỏ (heuristic và vét cạn n ≤ 10) và bị cắt khi quá 5.000 bước hoặc 2,5 MB; với vét cạn, phần trace bị cắt vẫn giữ bước kết quả cuối.
- Heuristic không bảo đảm tìm được nghiệm.
- Oracle độc lập chỉ khả thi với đồ thị nhỏ.
- Toàn bộ code do AI sinh. Tuy đã qua review chéo và test, vẫn có thể còn lỗi chưa phát hiện.

7.5. Hướng phát triển

- Thêm nhánh cận dùng chặn dưới từ clique và chặn trên từ DSATUR, để tìm χ(G) trên đồ thị lớn hơn.
- Thêm các bài toán biến thể: tô màu cạnh, tô màu có danh sách màu cho phép (list coloring).
- Cho phép nhập đồ thị bằng cách vẽ trực tiếp trên màn hình.

Chương 8. Phát triển mở rộng

Bài tô màu được xây trên một nền tảng chung. Nền tảng này cũng phục vụ **47 bài toán với 126 cách giải**, phần lớn theo sách Levitin. Một số cách giải không có trong sách và được ghi rõ trong app, ví dụ DSATUR, Hungarian.

Các thành phần dùng chung:

- registry tự nhận diện bài toán;
- Player;
- trang So sánh;
- trang Testcase;
- 12 kiểu hiển thị.

Nhóm	Ví dụ bài toán
Số học, tìm kiếm	Euclid, Fibonacci, tìm nhị phân, Strassen, khử Gauss
Sắp xếp, cây, băm	Quicksort, Mergesort, Heapsort, AVL, B-tree, băm
Đồ thị	DFS/BFS, Dijkstra, Floyd, Prim/Kruskal, luồng cực đại
Chuỗi, hình học, quy hoạch động	Horspool, Boyer-Moore, bao lồi, cái túi, simplex
Tối ưu tổ hợp	TSP, n-hậu, tổng tập con, nhân chuỗi ma trận, LCS

Danh mục đầy đủ ở Phụ lục B. **Phần mở rộng này không thay thế chiều sâu của bài trọng tâm.** Nó chỉ cho thấy cùng một khung Player, So sánh, Testcase dùng lại được cho nhiều bài toán khác.

Tài liệu tham khảo

1. A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed., Pearson, 2012.

2. D. J. A. Welsh, M. B. Powell, "An upper bound for the chromatic number of a graph and its application to timetabling problems", *The Computer Journal*, 10(1), 1967.

3. D. Brélaz, "New methods to color the vertices of a graph", *Communications of the ACM*, 22(4), 1979.

4. R. M. Karp, "Reducibility among combinatorial problems", 1972.

5. Tài liệu React, Vite, shadcn/ui, Playwright (bản trực tuyến).

Phụ lục

- **A.** Bảng 8 PR: merge commit, ngày, số vòng review, các lỗi chặn. Nguồn: `plans/reports/explore-261018-0006-ai-narrative-sources.md`.
- **B.** Danh mục 47 bài toán và 126 cách giải (trích từ `source-code/src/catalog/manifest.ts`).
- **C.** Danh sách testcase bài tô màu.
- **D.** Các prompt chính, nguyên văn, có ghi nguồn.